

Employee Performance Intelligence & Evaluation System

Development Prompt for Google AI Mode

1. Project Goal

Build a full-stack prototype called Employee Performance Intelligence & Evaluation System. The system evaluates employee performance using multiple metrics and generates an overall performance result. The project must be realistic for a 2-day development timeline and should not be a basic CRUD application.

2. Technology Stack

Frontend: React

UI Architecture: Atomic Design

State Management: Redux Toolkit

Routing: React Router

API Communication: Axios

Backend: Node.js + Express

Database: MongoDB

Charts: Recharts or another simple React chart library

MFE: Minimal; do not make MFE the main complexity.

Do not introduce unnecessary technologies.

3. Main Workflow

LOGIN → DASHBOARD → EMPLOYEE ID → EMPLOYEE DETAILS → SELECT REVIEW PERIOD → PERFORMANCE DATA → DATA VALIDATION → PERFORMANCE ANALYSIS → BENCHMARK ANALYSIS → TREND ANALYSIS → STRENGTH/WEAKNESS ANALYSIS → RECOMMENDATION → REVIEW → FINAL PERFORMANCE RESULT

4. Performance Metrics

Task Completion

Quality

Productivity

Attendance

Goal Achievement

Efficiency

Manager Feedback

Suggested weights:

Task Completion = 25%

Quality = 20%

Productivity = 15%

Attendance = 10%

Goal Achievement = 15%

Efficiency = 10%

Manager Feedback = 5%

5. Performance Rating

90–100: Outstanding

80–89: Excellent

70–79: Good

60–69: Needs Improvement

Below 60: Unsatisfactory

6. Analysis Engine

The backend should calculate:

- Overall performance score
- Performance rating
- Team average
- Difference from team average
- Performance trend
- Strengths
- Improvement areas
- Recommendations

Example rules:

- If quality ≥ 85 : identify strong quality performance.
- If efficiency < 70 : identify efficiency as an improvement area.
- If goal achievement < 70 : recommend focus on goal planning and achievement.
- If overall score ≥ 90 : recommend high-performer recognition.

Keep the rules simple and easy to understand.

7. Pages and Routes

Login — /login

Dashboard — /dashboard

Employee Search — /performance/start

Employee Details — /performance/employee

Review Period — /performance/period

Performance Data — /performance/data

Performance Analysis — /performance/analysis

Benchmark — /performance/benchmark

Insights — /performance/insights

Review — /performance/review

Final Result — /performance/result

8. Atomic Design

Atoms:

Button, Input, Label, Select, Loader, StatusBadge, ProgressBar, Score

Molecules:

EmployeeSearch, MetricCard, PerformanceMetric, ScoreCard, PerformanceStatus, DateRangeSelector

Organisms:

PerformanceHeader, EmployeeDetails, PerformanceMetrics, PerformanceChart, PerformanceAnalysis, PerformanceReview, PerformanceResult

Pages:

Use the pages listed above.

9. Redux Structure

Create:

authSlice

employeeSlice

performanceSlice

analysisSlice

uiSlice

Performance state should contain employee metrics and review information.

Analysis state should contain:

overallScore

rating

teamAverage

difference

trend

strengths

weaknesses

recommendations

10. Backend Structure

server/

src/

controllers/

services/

models/

routes/

middleware/

utils/

config/

app.js

server.js

Services:

performanceService.js

scoreCalculationService.js

benchmarkService.js

trendAnalysisService.js

recommendationService.js

11. MongoDB Collections

users:

_id, username, passwordHash, role

employees:

_id, employeeId, name, department, role, joiningDate

performanceRecords:

_id, employeeId, reviewPeriod, taskCompletion, quality, productivity, attendance, goalAchievement, efficiency, managerFeedback

performanceResults:

_id, employeeId, reviewPeriod, overallScore, rating, teamAverage, trend, strengths, weaknesses, recommendations

Use fictional test data only.

12. API

GET /api/employees/:employeeId

GET /api/performance/:employeeId

POST /api/performance/analyze

GET /api/performance/:employeeId/trend

GET /api/performance/:employeeId/benchmark

GET /api/performance/:employeeId/result

13. Development Rules for Google AI Mode

I am a beginner/intermediate developer and need to understand the code.

Do NOT dump the entire project at once.

Work step-by-step.

First provide only:

1. Complete architecture
2. Folder structure
3. Required software
4. Required npm packages
5. Installation commands
6. Environment variables
7. Development order

Then wait for me to say NEXT.

For every code file:

- Clearly show the file path.
- Give complete code for that file.
- Briefly explain what it does.
- Explain important code sections.
- Give the exact command to run.
- Explain the expected output.
- Help debug errors before moving forward.

Prioritize a working application over unnecessary complexity.

Do not use real employee personal data.

Start with architecture, folder structure, packages, installation commands, and development plan only. Do not generate application code yet.